



Universidad Nacional Autónoma de México



Facultad de Ingeniería

Proyecto final:

Implementacion en Raspberry pi pico 2W del juego de
Galaga inalambrico.

Integrantes:

Espinoza Matamoros Percival Ulises - 320025561

Flores Colin Victor Jaziel - 320266083

García Cortés Adolfo de Jesús - 320317264

Lara Hernandez Angel Husiel - 320060829

Lugo Manzano Rodrigo - 320206968

Microcomputadoras

Grupo: 03 - Semestre: 2026-2

Calf: _____



1. Objetivo:

Diseñar e implementar un videojuego estilo Galaga completamente funcional sobre una Raspberry Pi Pico 2W programada en MicroPython, capaz de ejecutarse en tiempo real sobre una pantalla TFT ILI9341 de 320x240 pixeles, siendo controlado de forma inalámbrica mediante un segundo microcontrolador Pico 2W que actúa como mando Bluetooth Low Energy (BLE) equipado con una unidad de medición inercial (IMU) MPU-6050 y dos botones físicos uno para disparar y otro para poner pausa. El sistema combina varios componentes de hardware y software para aprovechar al máximo las capacidades del microcontrolador. Utiliza comunicación SPI para mostrar gráficos rápidamente en la pantalla, protocolo I2C para leer datos del sensor inercial, Bluetooth Low Energy para la conexión inalámbrica, y PWM para generar sonidos con un buzzer. Además, emplea programación asíncrona con `uasyncio` para que todas las funciones del juego trabajen al mismo tiempo de manera organizada y eficiente.

2. Desarrollo:

2.1. Descripción General del Sistema

El proyecto se compone de dos unidades físicas independientes que se comunican de forma inalámbrica: la Consola, que contiene la pantalla TFT y el buzzer, y el Mando, que integra el sensor IMU y los botones de control. La arquitectura de software sigue un modelo cliente-servidor BLE donde el Mando actúa como periférico (servidor GATT) y la Consola actúa como central (cliente GATT).

Tabla 1: Componentes de hardware del sistema

Módulo	Componente	Interfaz	Función
Consola	Raspberry Pi Pico 2W	—	CPU principal del juego
Consola	TFT ILI9341 320x240	SPI0 (40 MHz)	Pantalla de juego
Consola	Buzzer Piezoeléctrico	PWM (GP15)	Efectos de sonido
Mando	Raspberry Pi Pico 2W	—	CPU del controlador
Mando	MPU-6050 (IMU)	I2C0 (400 kHz)	Control de movimiento
Mando	Botón Disparo (GP4)	GPIO Pull-up	Disparar proyectil
Mando	Botón Pausa (GP5)	GPIO Pull-up	Pausar / menú



2.2. Arquitectura de Software

El software de la Consola se organiza en módulos independientes que encapsulan la lógica de cada entidad del juego. El punto de entrada `main.py` lanza dos tareas asíncronas concurrentes mediante `uasyncio`: `ble_central_task()` para mantener la conexión con el mando, y `game_task()` para ejecutar el bucle principal del juego. Esta arquitectura garantiza que la pérdida temporal de conexión BLE no congele la lógica de juego.

Tabla 2: Módulos de software de la Consola

Módulo	Responsabilidad
<code>main.py</code>	Inicialización de hardware (SPI, display, buzzer), lanzamiento de tareas <i>async</i> BLE y juego.
<code>game.py</code>	Clase Game : bucle principal, lógica de juego, detección de colisiones, renderizado, gestión de niveles y menús.
<code>player.py</code>	Clase Player : posición, movimiento analógico basado en IMU, dibujo del sprite y animación de explosión.
<code>enemy.py</code>	Clase Enemy : posición en formación, animación alternada de sprites, puntuación por fila y explosión.
<code>asteroid.py</code>	Clases Asteroid , Guardian y DiveBomber : enemigos especiales con comportamientos de movimiento propios.
<code>bullet.py</code>	Clase Bullet : proyectiles del jugador y enemigos con velocidad, color y límites de pantalla configurables.
<code>buzzer.py</code>	Clase Buzzer : cola de tonos PWM no bloqueante con melodías y efectos de sonido para cada evento.
<code>virtual_button.py</code>	Clase VirtualButton : desacopla la lógica de pulsación (<i>pressed</i> / <i>held</i>) del estado BLE recibido.
<code>colours.py</code>	Constantes de colores en formato RGB565 y paleta de colores por nivel.

2.2.1. Módulo `main.py`: Control Principal y Conexión

El archivo `main.py` se encarga de encender todas las partes del hardware y controlar cómo funciona la aplicación usando tareas simultáneas (programación asíncrona) con `uasyncio`. Su trabajo principal es preparar los componentes físicos, crear dos tareas que funcionan al



mismo tiempo y arrancarlas juntas con `asyncio.gather()`. Esto asegura que la comunicación por Bluetooth (BLE) y el juego operen en equipo sin estorbarse mutuamente.

2.2.2. Preparación del Hardware

La consola prepara la pantalla para que actualice rápido (a 40 MHz) y configura la pequeña bocina (buzzer) en el pin GP15. También crea un objeto llamado `ControllerState` que funciona como una memoria compartida: la tarea de Bluetooth guarda ahí los datos que recibe del mando, y el juego los lee constantemente sin tener problemas, ya que MicroPython organiza muy bien las tareas.

```
1 from machine import Pin, SPI
2 from ili9341 import Display
3
4 SPI_ID, PIN_SCK, PIN_MOSI, PIN_MISO = 0, 18, 19, 16
5 PIN_CS, PIN_DC, PIN_RST, BUZZER_PIN = 17, 20, 21, 15
6
7 spi = SPI(SPI_ID, baudrate=40_000_000, polarity=0, phase=0,
8           sck=Pin(PIN_SCK), mosi=Pin(PIN_MOSI), miso=Pin(PIN_MISO))
9
10 display = Display(spi,
11                  cs=Pin(PIN_CS, Pin.OUT),
12                  dc=Pin(PIN_DC, Pin.OUT),
13                  rst=Pin(PIN_RST, Pin.OUT),
14                  width=320, height=240, rotation=270)
15 display.clear()
```

Listing 1: Inicialización de hardware SPI y display en la Consola

En el mando de juego, se configura la conexión I2C0 de forma rápida (400 kHz) para comunicarse con el sensor de movimiento (MPU-6050) y se preparan los botones aprovechando las conexiones internas del RP2350. Hacer esto rápido evita que leer el sensor haga lento el programa, sobre todo porque esta lectura se hace cada 10 milisegundos.

```
1 from machine import Pin, I2C
2 from imu import MPU6050
3
4 # Bus I2C0: SDA=GP8, SCL=GP9, frecuencia 400 kHz (Fast Mode)
5 i2c = I2C(0, sda=Pin(8), scl=Pin(9), freq=400000)
6 imu = MPU6050(i2c)
```



```
7
8 # Pull-up interno: Pin.value() == 0 significa presionado
9 btn_fire = Pin(4, Pin.IN, Pin.PULL_UP)
10 btn_pause = Pin(5, Pin.IN, Pin.PULL_UP)
```

Listing 2: Inicialización del bus I2C y los botones en el Mando

2.2.3. Empaquetado y Envío de Datos por Bluetooth

Para enviar los datos del control a la consola de forma rápida, se comprimen en un paquete muy pequeño de 10 bytes usando `struct.pack`. Ahí se guardan dos números con decimales (inclinación X e Y) y dos números enteros (para saber si se apretaron los botones). Hacer esto ocupa muy poco espacio, mucho menos que usar formatos de texto largos. Esta información se manda unas 100 veces por segundo, y además el programa limpia la memoria periódicamente con `gc.collect()` para que la tarjeta no se trabe tras un rato de uso.

```
1 import struct, gc
2
3 async def peripheral_task():
4     while True:
5         connection = await aioble.advertise(
6             250_000, name='PicoMando', services=[_SERVICE_UUID]
7         )
8         while connection.is_connected():
9             ax = imu.accel.x    # Inclinacion lateral
10            ay = imu.accel.y    # Inclinacion frontal
11            fire_pressed = (btn_fire.value() == 0)
12            pause_pressed = (btn_pause.value() == 0)
13
14            # '<ffbb': 2 float32 (8 bytes) + 2 uint8 (2 bytes) = 10
15            bytes
16            payload = struct.pack('<ffbb', ax, ay,
17                                   int(fire_pressed), int(pause_pressed)
18            )
19            _char.write(payload)
20            await _char.notify(connection)
21
22            gc.collect()          # Liberar heap: evita
23            fragmentacion
24            await asyncio.sleep_ms(10) # Tasa de envio: 100 Hz
```



Listing 3: Lectura del IMU, empaquetado y notificación BLE en el Mando

2.2.4. Recepción de Datos en la Consola

La tarea `ble_central_task()` busca el control llamado 'PicoMando' durante 5 segundos. Cuando lo encuentra, se conecta y se queda atenta. Cada vez que llega información, la desempaqueta y actualiza las acciones en el juego. Si por algún motivo se pierde la conexión, el programa espera 2 segundos y se vuelve a conectar solo, sin interrumpir tu partida.

```
1  async def ble_central_task():
2      while True:
3          try:
4              async with aioble.scan(5000, interval_us=30000,
5                                     window_us=30000, active=True) as
6
7                  scanner:
8                      async for result in scanner:
9                          if result.name() == 'PicoMando':
10                             device = result.device
11                             break
12                         else:
13                             continue
14
15                     connection = await device.connect(timeout_ms=2000)
16                     service      = await connection.service(_SERVICE_UUID)
17                     char          = await service.characteristic(_CHAR_UUID)
18                     await char.subscribe(notify=True)
19                     while True:
20                         data = await char.notified()
21                         # Deserializar: '<ffbb' -> ax, ay, fire, pause
22                         x, y, fire_val, pause_val = struct.unpack('<ffbb', data
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
259
```



2.3. Módulo game.py: Corazón del Juego

Este archivo tiene más de 600 líneas porque es la parte principal. Aquí se controla todo: cómo dibujan los gráficos, cómo se mueven los enemigos, los choques entre naves y los menús. La clase `Game` agrupa todas las piezas físicas (pantalla, botones y sonido) para manejarlas en un solo lugar.

2.3.1. Preparación y Estado del Juego

Al empezar, se crean todos los personajes y objetos, además de preparar las reglas: dónde empiezan los enemigos, cuántas vidas tienes, los puntos, el nivel y la dificultad. La función `setup_game()` acomoda a los 24 enemigos en sus posiciones iniciales de 3x8 y reinicia el fondo cada vez que empiezas una nueva partida.

```
1 class Game:
2     def __init__(self, display, buzzer, btn_fire, btn_pause, ctrl_state
3         ):
4         self.display      = display
5         self.buzzer       = buzzer
6         self.btn_fire     = btn_fire
7         self.btn_pause    = btn_pause
8         self.ctrl_state   = ctrl_state
9
10        self.player       = Player(WIDTH, HEIGHT)
11        self.asteroid      = Asteroid(WIDTH, HEIGHT)
12        self.guardian      = Guardian(WIDTH, HEIGHT)
13        self.dive_bomber   = DiveBomber(WIDTH, HEIGHT)
14
15        self.enemies       = []      # Lista de 24 instancias Enemy
16        self.player_bullets = []      # Projectiles del jugador activos
17        self.enemy_bullets = []      # Projectiles enemigos activos
18        self.stars          = []      # Campo de 30 estrellas
19
20        self.player_lives   = 3
21        self.total_score    = 0
22        self.level         = 1
23        self.difficulty_mode = 1      # 0=Facil, 1=Normal, 2=Difícil
24        self.enemy_dir      = 1      # 1=derecha, -1=izquierda
25        self.current_row    = 0      # Fila activa del movimiento
26        rotativo
```



Listing 5: Constructor de la clase Game: instanciación de entidades y estado inicial

2.3.2. Ciclo Principal del Juego

La función `run()` es el ciclo infinito que mantiene el juego vivo. Se encarga de mostrar las diferentes pantallas: inicio, puntuaciones, la partida en sí, pausa y "Fin de juego". Cuando estás jugando, la pantalla se actualiza unas 30 veces por segundo. En cada paso mueve al jugador, procesa los disparos, mueve a los enemigos, revisa choques y hace los sonidos. Si logras derrotar a todos, te avanza al siguiente nivel.

```
1  async def run(self):
2      while True:                                # Ciclo externo: reinicia la
partida
3          self.setup_game()
4          await self._splash_screen()             # Pantalla de credits
5          await self._info_screen()              # Tabla de puntuaciones
6          self.buzzer.play_start()
7          self.display.clear(BLACK)
8          self._save_positions()                 # Inicializar dirty-rect
9
10         while True:                             # Ciclo interno: partida
activa
11             self.btn_fire.update_state(self.ctrl_state.fire_raw)
12             self.btn_pause.update_state(self.ctrl_state.pause_raw)
13
14             if self.btn_pause.is_pressed():
15                 await self._pause()             # Menu de pausa
16
17             self._save_positions()              # Guardar para dirty-rect
18             self.player.move_analog(-self.ctrl_state.x, -self.
ctrl_state.y)
19
20             # Disparar: pulsacion simple o automatico al mantener
21             if self.btn_fire.is_pressed():
22                 self.player_bullets.append(
23                     Bullet(self.player.x + PLAYER_W//2, self.player.y,
-5, CYAN, HEIGHT))
24                 self.buzzer.play_shoot()
```



```
25         self._update_bullets()
26         self._collision_detection()
27         self._maybe_enemy_shoot()
28         self.asteroid.update(self.frame_count, self.
29         asteroid_interval, self.asteroid_speed)
30         self.guardian.update(self.frame_count)
31         self.dive_bomber.update(self.frame_count, self.
32         dive_bomber_interval,
33                                     self.dive_bomber_speed, self.
34         enemies)
35         self._move_enemies()
36         self._render() # Renderizado diferencial
37
38         # Limpiar proyectiles inactivos para liberar memoria
39         self.player_bullets = [b for b in self.player_bullets if b.
40         active]
41         self.enemy_bullets = [b for b in self.enemy_bullets if b.
42         active]
43
44         self.buzzer.update() # Avanzar cola de audio
45         self.frame_count += 1
46
47         if self._count_alive() == 0: await self._next_level()
48         if self._is_game_over(): break
49         await asyncio.sleep_ms(33) # ~30 fps
```

Listing 6: Estructura del bucle principal de juego en run()

2.3.3. Dibujado Inteligente de Pantalla

Para que la pantalla no parpadee ni se ponga lenta, el juego no borra y dibuja toda la pantalla en cada paso. En vez de eso, recuerda dónde estaban los personajes, calcula cuánto se movieron, y solo borra la pequeña estela que dejaron atrás con `fill_hrect()`. Esto ahorra mucha memoria y esfuerzo al procesador de la pantalla.

```
1 def _render(self):
2     fh = self.display.fill_hrect
3     fc = self.frame_count
4
```



```
5 self._update_and_draw_stars() # Actualizar fondo en primer plano
6
7 # Calcular desplazamiento del jugador respecto al frame anterior
8 dx = self.player.x - self.player._px # Delta horizontal
9 dy = self.player.y - self.player._py # Delta vertical
10
11 # Borrar solo la estela horizontal expuesta
12 if dx > 0:
13     fh(self.player._px, self.player._py, dx, PLAYER_H, BLACK)
14 elif dx < 0:
15     fh(self.player.x + PLAYER_W, self.player._py, -dx, PLAYER_H,
16     BLACK)
17
18 # Borrar solo la estela vertical expuesta
19 if dy > 0:
20     fh(self.player._px, self.player._py, PLAYER_W, dy, BLACK)
21 elif dy < 0:
22     fh(self.player._px, self.player.y + PLAYER_H, PLAYER_W, -dy,
23     BLACK)
24
25 self.player.draw(self.display) # Redibujar sprite en nueva
26 posicion
```

Listing 7: Renderizado diferencial del jugador: borrado de estela por delta de posición

2.3.4. Fondo de Estrellas Movable y Brillante

El fondo tiene 30 estrellas que caen a diferentes velocidades, lo que da una ilusión de profundidad (como si unas estuvieran más lejos que otras). Para darle un toque realista, usan operaciones matemáticas rápidas para hacer que el brillo de algunas estrellas cambie de repente, haciendo que parezcan titilar sin gastar recursos de la tarjeta.

```
1 def _update_and_draw_stars(self):
2     dp = self.display.draw_pixel # Referencia local: 4x mas rapido que
3     global
4     for s in self.stars:
5         dp(s[0], s[1], BLACK) # 1. Borrar posicion anterior
6         s[1] += s[2] # 2. Desplazar verticalmente (
7         paralaje)
8         if s[1] >= HEIGHT: # 3. Si sale de pantalla, reiniciar
```



```
arriba
7         s[1] = 0
8         s[0] = randint(0, WIDTH - 1)
9         s[3] = 8 + (getrandbits(5) % 24)
10
11         # 4. Efecto de titileo: cambio aleatorio de brillo (prob. 1/64)
12         if getrandbits(6) == 0:
13             flicker = (getrandbits(3) % 7) - 3 # Rango: -3 a +3
14             b = max(0, min(s[3] + flicker, 31)) # Clampear 0-31
15             s[4] = self._gray565(b * 8) # Recalcular RGB565
16
17         dp(s[0], s[1], s[4]) # 5. Dibujar en nueva posicion
```

Listing 8: Actualización y dibujo del campo de estrellas con titileo aleatorio

2.3.5. Movimiento de los Enemigos

El grupo de 24 enemigos se mueve por filas. En vez de mover a todos al mismo tiempo, mueve una sola fila por turno. Conforme vas destruyendo enemigos, el resto se empieza a mover más y más rápido (igual que en los juegos clásicos de maquinita). Si un enemigo toca el borde de la pantalla, todo el grupo baja un poco y empiezan a moverse hacia el lado contrario.

```
1 def _move_enemies(self):
2     total = len(self.enemies)
3     alive = self._count_alive()
4     # Delay inversamente proporcional a enemigos vivos: la formacion
5     acelera
6     delay = max(1, alive * 4 // total)
7     if self.frame_count - self.last_enemy_move < delay:
8         return
9     self.last_enemy_move = self.frame_count
10
11     # Verificar si la fila activa alcanza el borde de la pantalla
12     can_move = True
13     for e in self.enemies:
14         if e.alive and e.row == self.current_row:
15             if self.enemy_dir > 0 and e.x + ALIEN_W + self.enemy_speed
16             >= WIDTH:
17                 can_move = False
18             elif self.enemy_dir < 0 and e.x - self.enemy_speed < 0:
```



```
17         can_move = False
18
19     if not can_move:                # Invertir direccion y descender toda
    la formacion
20         self.enemy_dir *= -1
21         for e in self.enemies:
22             if e.alive: e.y += ENEMY_DROP    # ENEMY_DROP = 10 pixeles
23
24     for e in self.enemies:          # Mover solo la fila activa
25         if e.alive and e.row == self.current_row:
26             e.x += self.enemy_dir * self.enemy_speed
27
28     self.current_row = (self.current_row + 1) % NUM_ROWS
```

Listing 9: Movimiento rotativo por filas y descenso al alcanzar el borde

2.3.6. Choques entre Objetos

El juego sabe cuando dos cosas chocan al revisar si sus coordenadas se enciman al mismo tiempo. Todo el tiempo revisa si tus disparos le dan a los enemigos, o si los disparos de ellos te dan a ti. Como un detalle extra, hay un enemigo llamado "Guardian" que si chocas con él no te hace daño, sino que te regala una vida extra.

```
1 def _collision_detection(self):
2     # 1. Proyectiles del jugador vs formacion de enemigos
3     for e in self.enemies:
4         for b in self.player_bullets:
5             if (b.active and e.alive and
6                 b.x >= e.x          and b.x <= e.x + ALIEN_W and
7                 b.y >= e.y          and b.y <= e.y + ALIEN_H):
8                 e.alive = False
9                 e.explode()          # Activar animacion de
    explosion
10                 self.total_score += e.score    # 30, 20 o 10 segun fila
11                 self.buzzer.play_explosion()
12                 b.active = False
13
14     # 2. Guardian vs jugador: power-up (+1 vida, max 3)
15     if (self.guardian.active and
16         self.guardian.x >= self.player.x and
```



```
17     self.guardian.x <= self.player.x + PLAYER_W and
18     self.guardian.y >= self.player.y and
19     self.guardian.y <= self.player.y + PLAYER_H):
20     self.guardian.active = False
21     if self.player_lives < 3:
22         self.player_lives += 1
23     self.buzzer.play_powerup()
24
25     # 3. Projectiles enemigos vs jugador
26     for b in self.enemy_bullets:
27         if (b.active and
28             b.x >= self.player.x and b.x <= self.player.x + PLAYER_W
29             and
30             b.y >= self.player.y and b.y <= self.player.y + PLAYER_H):
31             b.active = False
32             self.player_lives -= 1
33             self.player.explode()
34             self.buzzer.play_killed()
```

Listing 10: Detección AABB: proyectiles del jugador contra formación enemiga

2.3.7. Disparos Sorpresa de los Enemigos

Cada cierto tiempo, el juego escoge a un enemigo vivo al azar para que te lance un disparo naranja hacia abajo. Conforme avanzas de nivel, este tiempo de espera se hace más corto, haciendo que tengas que esquivar más cosas.

```
1 def _maybe_enemy_shoot(self):
2     alive = self._count_alive()
3     if alive == 0: return
4     # Esperar el intervalo de cadencia antes del siguiente disparo
5     if self.frame_count - self.last_enemy_shot < self.
6     enemy_rate_of_fire:
7         return
8     self.last_enemy_shot = self.frame_count
9
10    alive_list = [e for e in self.enemies if e.alive]
11    e = alive_list[randint(0, len(alive_list) - 1)] # Enemigo
12    aleatorio
13    # Proyectil desde el centro inferior del sprite del enemigo
14    self.enemy_bullets.append(
```



```
13     Bullet(e.x + ALIEN_W // 2, e.y + ALIEN_H, 4, ORANGE, HEIGHT))
14     self.buzzer.play_enemy_shoot()
```

Listing 11: Disparo aleatorio de los enemigos de la formación

2.3.8. Pantallas y Menús

El juego maneja cinco pantallas diferentes. Tienes una de créditos y una donde te enseñan cuántos puntos vale cada enemigo. El menú de pausa es bastante interactivo: te puedes mover por las opciones inclinando tu control (leyendo el sensor) e incluso puedes cambiar la dificultad sin tener que reiniciar la partida.

```
1  async def _pause(self):
2      options      = ['Continuar', 'Modo', 'Salir al menu']
3      mode_names   = ['Facil', 'Normal', 'Dificil']
4      selected, last_move_time = 0, 0
5
6      while True:
7          now = time.ticks_ms()
8          self.btn_fire.update_state(self.ctrl_state.fire_raw)
9          fuerza_x = -self.ctrl_state.x  # Eje X del acelerometro
10
11         # Navegar con inclinacion: umbral 0.4g, debounce de 300 ms
12         if time.ticks_diff(now, last_move_time) > 300:
13             if fuerza_x > 0.4:
14                 selected = (selected + 1) % len(options)
15                 last_move_time = now
16             elif fuerza_x < -0.4:
17                 selected = (selected - 1) % len(options)
18                 last_move_time = now
19
20         if self.btn_fire.is_pressed():
21             if selected == 0: break          # Continuar
22             elif selected == 1:              # Cambiar
23                 dificultad
24                 self.difficulty_mode = (self.difficulty_mode + 1) % 3
25                 self._set_difficulty()      # Aplicar en
26                 caliente
27             elif selected == 2:              # Salir al
28                 menu
```



```
26         self.exit_to_menu = True
27         break
28     await asyncio.sleep_ms(50)
```

Listing 12: Menú de pausa: navegación por inclinación del mando

2.3.9. Barra de Información

La pantalla siempre muestra tus puntos y en qué nivel vas en la parte de arriba. Dependiendo del nivel, los colores de los textos cambian para que se vea diferente. Arriba a la derecha aparecen dibujadas tus vidas como pequeñas naves. Para que la pantalla funcione más rápido, esa parte solo se actualiza cuando te quitan una vida.

```
1 def _update_status(self):
2     score_text = '{:05d} LVL {:2d}'.format(self.total_score, self.level)
3     color = LEVEL_COLOURS[(self.level - 1) % 10] # Color cambia por
4     nivel
5     self.display.draw_text8x8(5, 5, score_text, color, BLACK)
6
7     # Borrar area de vidas solo si el conteo bajo (evita residuos SPI)
8     if self.player_lives < self._prev_lives:
9         self.display.fill_hrect(WIDTH - 100, 0, 100, 35, BLACK)
10        self._prev_lives = self.player_lives
11
12    # Dibujar sprite de nave por cada vida restante (de derecha a
13    izquierda)
14    for idx in range(self.player_lives):
15        x_pos = WIDTH - ((idx + 1) * (PLAYER_W + 5))
16        self.display.draw_sprite(player_sprite, x_pos, 5, PLAYER_W,
17        PLAYER_H)
```

Listing 13: Actualización del HUD: puntaje, nivel y vidas

2.4. Módulo player.py: Movimiento del Jugador

Aquí se maneja la posición y el movimiento de tu nave. Te mueves dependiendo de cómo inclines el control: inclinarlo adelante o atrás te mueve de forma horizontal, y a los lados de forma vertical (solo en la mitad inferior de la pantalla). Hay un pequeño "filtro" del 15% para ignorar movimientos muy leves, así no te mueves por accidente si te tiembla la mano.



```
1 def move_analog(self, force_x, force_y):
2     mov_x = int(force_x * 15)    # Escala: 1g -> 15 pixeles de
3     mov_y = int(force_y * 15)    desplazamiento
4
5     if abs(force_y) > 0.15:      # Zona muerta del 15%: ignorar deriva
6         self.x -= mov_y
7         if self.x < 0:            self.x = 0
8         if self.x > self._width - PLAYER_W: self.x = self._width -
PLAYER_W
9
10    if abs(force_x) > 0.15:
11        self.y -= mov_x
12        if self.y < self._width // 2 - 40: # Limite superior: mitad de
13        pantalla
14        self.y = self._width // 2 - 40
15        if self.y > 240 - PLAYER_H:      # Limite inferior: borde de
16        pantalla
17        self.y = 240 - PLAYER_H
```

Listing 14: Movimiento analógico del jugador con zona muerta

2.5. Módulo enemy.py: Animación de los Enemigos

Este archivo maneja a cada enemigo del juego. Para que parezca que están vivos, alternan entre dos imágenes diferentes muy rápido. Usan un truco matemático muy eficiente para hacerlo sin que el microcontrolador gaste energía de más. También controlan que los enemigos de hasta arriba valgan 30 puntos, y los de más abajo valgan 10, tal como en los juegos viejos.

```
1 _ALIEN_PAIRS = ((alien1, alien2), (alien3, alien4), (alien5, alien6))
2 _SCORES = (30, 20, 10)    # Puntuacion segun fila
3
4 class Enemy:
5     def __init__(self, start_x, start_y, row):
6         self.x      = start_x
7         self.y      = start_y
8         self.alive  = True
9         self.row    = row
10        self.score   = _SCORES[row % 3]
```



```
11         self.explode_count = 0
12
13     def draw(self, display, frame_count):
14         if self.alive:
15             pair = _ALIEN_PAIRS[self.row % 3]
16             # frame_count & 0x8: alterna cada 8 frames sin division ni
17             modulo
18             sprite = pair[0] if frame_count & 0x8 else pair[1]
19             display.draw_sprite(sprite, self.x, self.y, ALIEN_W,
20                                ALIEN_H)
21         elif self.explode_count:
22             explosion = (explode2, explode2, explode1, explode1)
23             display.draw_sprite(explosion[self.explode_count - 1],
24                                self.x, self.y, EXPLODE_W, EXPLODE_H)
25             self.explode_count -= 1
```

Listing 15: Animación de sprite por enmascaramiento de bits y puntuación por fila

2.6. Módulo asteroid.py: Enemigos Especiales

Aquí se definen tres enemigos extra con formas de moverse muy distintas al resto. El "Asteroid" baja casi en línea recta, el "Guardian" también baja pero es bueno porque te regala vidas, y el "DiveBomber" baja haciendo curvas hacia los lados, lo que lo hace muy difícil de esquivar.

```
1 import math
2
3 def update(self, frame_count, interval, speed, enemies):
4     if not self.active and frame_count - self.last_spawn > interval:
5         # Escoger un enemigo vivo de la fila 2 como punto de
6         lanzamiento
7         row2 = [e for e in enemies if e.alive and e.row == 2]
8         if row2:
9             e = row2[randint(0, len(row2) - 1)]
10            self.x = e.x
11            self.y = e.y
12            self.active = True
13            self.last_spawn = frame_count
14
15            self.y += speed # Descender verticalmente
16            # Componente X sinusoidal: 3 ciclos completos a lo largo de la
```



```
altura
16     self.x = self._width // 2 + int(
17         20 * math.sin(3 * 2 * math.pi * self.y / self._height)
18     )
19     if self.y > self._height:
20         self.active = False
```

Listing 16: Trayectoria sinusoidal del DiveBomber

2.7. Módulo bullet.py: Disparos

Esta parte maneja los disparos. Básicamente son iguales para todos, solo que los tuyos son azules y van para arriba, y los de los enemigos son naranjas y van para abajo. El programa los borra en cuanto salen de la pantalla para mantener todo limpio y no saturar la memoria.

```
1 class Bullet:
2     def __init__(self, x, y, dy, color, height):
3         self.x      = x
4         self.y      = y
5         self.dy     = dy      # Negativo: sube (jugador); Positivo:
6         baja (enemigo)
7         self.active = True
8         self.color  = color   # CYAN=jugador, ORANGE=enemigo
9         self._height = height
10
11     def update(self):
12         self.y += self.dy
13         # Desactivar al salir de pantalla: el bucle lo eliminara de la
14         lista
15         if self.y < 0 or self.y > self._height:
16             self.active = False
17
18     def draw(self, display):
19         if self.active:
20             display.fill_hrect(self.x, self.y, 2, 4, self.color)
```

Listing 17: Ciclo de vida del proyectil: movimiento y desactivación por límites



2.8. Módulo buzzer.py: Sonidos y Música

Este archivo hace sonar a la pequeña bocina que está conectada. En vez de pausar el juego para que suene algo, funciona como una "fila de espera": puedes decirle que haga varios ruidos seguidos y los va a hacer en orden, uno tras otro. Siempre está configurado para sonar al máximo volumen posible para que se escuche bien claro.

```
1 def update(self):
2     now = time.ticks_ms()
3     if self._current is not None:
4         freq, dur = self._current
5         # Verificar si la nota activa cumplio su duracion
6         if time.ticks_diff(now, self._start_time) >= dur:
7             self._stop_pwm()
8             self._current = None
9             return # Esperar a que termine antes de pasar a la siguiente
10
11     if self._queue: # Iniciar la siguiente nota de la cola
12         self._current = self._queue.pop(0)
13         freq, dur = self._current
14         self._start_time = now
15         if freq > 0:
16             self._start_pwm(freq)
17
18 def _start_pwm(self, freq):
19     if self.pwm is not None: self.pwm.deinit()
20     self.pwm = PWM(Pin(self.pin_num))
21     self.pwm.freq(freq)
22     self.pwm.duty_u16(512) # Ciclo de trabajo del 50%: maximo volumen
23
24 # Melodia de muerte del jugador: escala descendente
25 def play_killed(self):
26     for f in [1046, 988, 880, 784, 698, 659, 587]:
27         self.tone(f, 80)
```

Listing 18: Método update(): gestión de cola no bloqueante y control de duración



2.9. Módulo `virtual_button.py`: Control de Botones

Esto es un pequeño filtro que le ayuda al programa a saber exactamente qué quieres hacer con los botones. Puede notar la diferencia entre apretarlo rápido una vez, o dejarlo presionado mucho tiempo (por ejemplo para que la nave dispare de forma automática). Esto es súper útil porque evita tener que poner más componentes físicos y hace que se sienta más natural jugarlo.

```
1 class VirtualButton:
2     def __init__(self):
3         self._is_pressed = False
4         self._was_pressed = False
5         self._held_counter = 0
6
7     def update_state(self, is_pressed_bool):
8         self._was_pressed = self._is_pressed
9         self._is_pressed = is_pressed_bool
10        if self._is_pressed: self._held_counter += 1
11        else: self._held_counter = 0
12
13        def is_pressed(self): # True solo en el frame del flanco
14        ascendente
15        event = self._is_pressed and not self._was_pressed
16        if event: self._was_pressed = True
17        return event
18
19        def is_held(self): # True si lleva mas de 5 frames presionado
20        return self._is_pressed and self._held_counter > 5
```

Listing 19: Detección de flanco ascendente y estado mantenido en `VirtualButton`

2.10. Módulo `colours.py`: Colores

Aquí están guardados todos los colores que se pueden usar. Están en un formato numérico especial que la pantalla lee de golpe, lo que le ahorra cálculos difíciles al procesador mientras juegas. También hay una paleta con 10 colores listos para que la información del juego cambie de color a medida que pasas de nivel.

```
1 # Formato RGB565: 5 bits rojo / 6 bits verde / 5 bits azul
2 BLACK = 0x0000 # 0000 0000 0000 0000
```

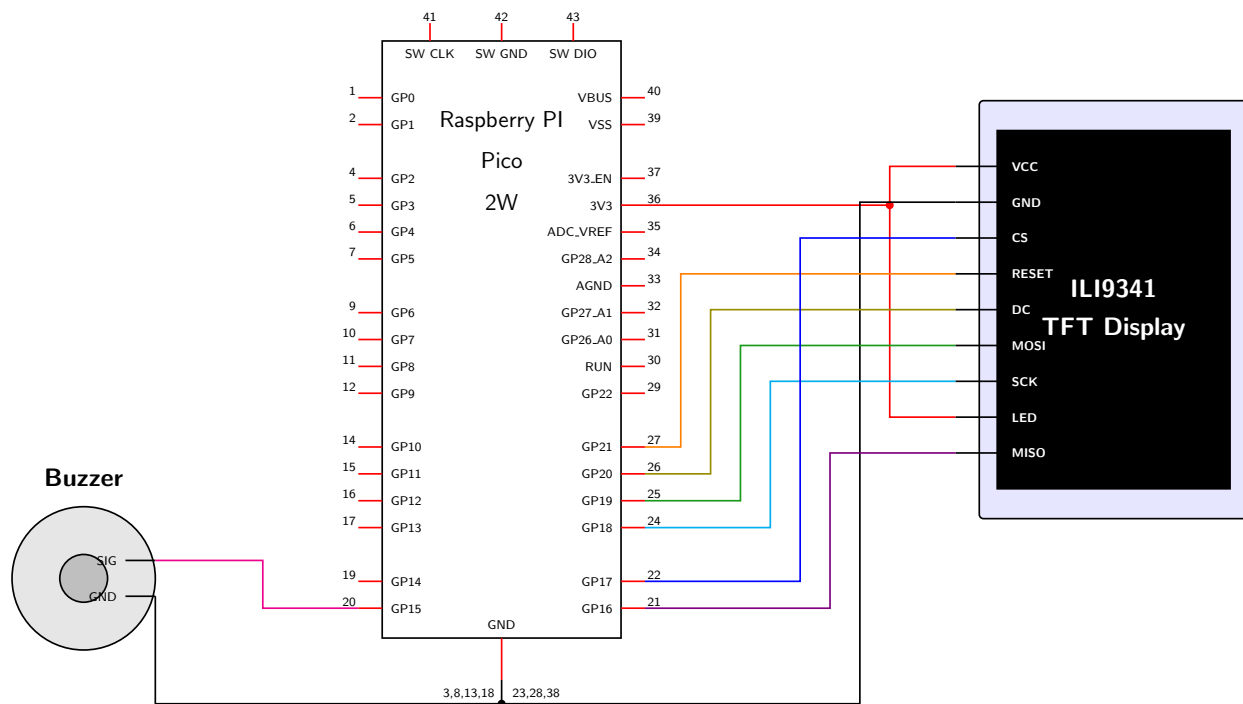
```

3 RED      = 0xF800    # 1111 1000 0000 0000
4 LIME     = 0x07E0    # 0000 0111 1110 0000
5 BLUE     = 0x001F    # 0000 0000 0001 1111
6 WHITE    = 0xFFFF    # 1111 1111 1111 1111
7 CYAN     = 0x07FF    # Color proyectiles del jugador
8 ORANGE   = 0xFD20    # Color proyectiles enemigos
9 YELLOW   = 0xFFE0    # Texto de interfaz
10
11 # Paleta de 10 colores: el HUD cambia de color en cada nivel
12 LEVEL_COLOURS = [MAROON, RED, ORANGE, YELLOW, GREEN,
13                  LIME, AQUA, BLUE, NAVY, PURPLE]

```

Listing 20: Constantes de color RGB565 y paleta por nivel

3. Diagrama de conexiones del modulo principal



La consola concentra la pantalla TFT ILI9341 y el buzzer piezoeléctrico. La pantalla se conecta al bus SPI0 de la Pico 2W usando cuatro pines de datos (MOSI, MISO, SCK, CS) más dos pines de control (DC y RST). El buzzer se controla mediante la salida PWM del pin GP15. Tanto la pantalla como el buzzer comparten el plano de tierra de la Pico, y la pantalla



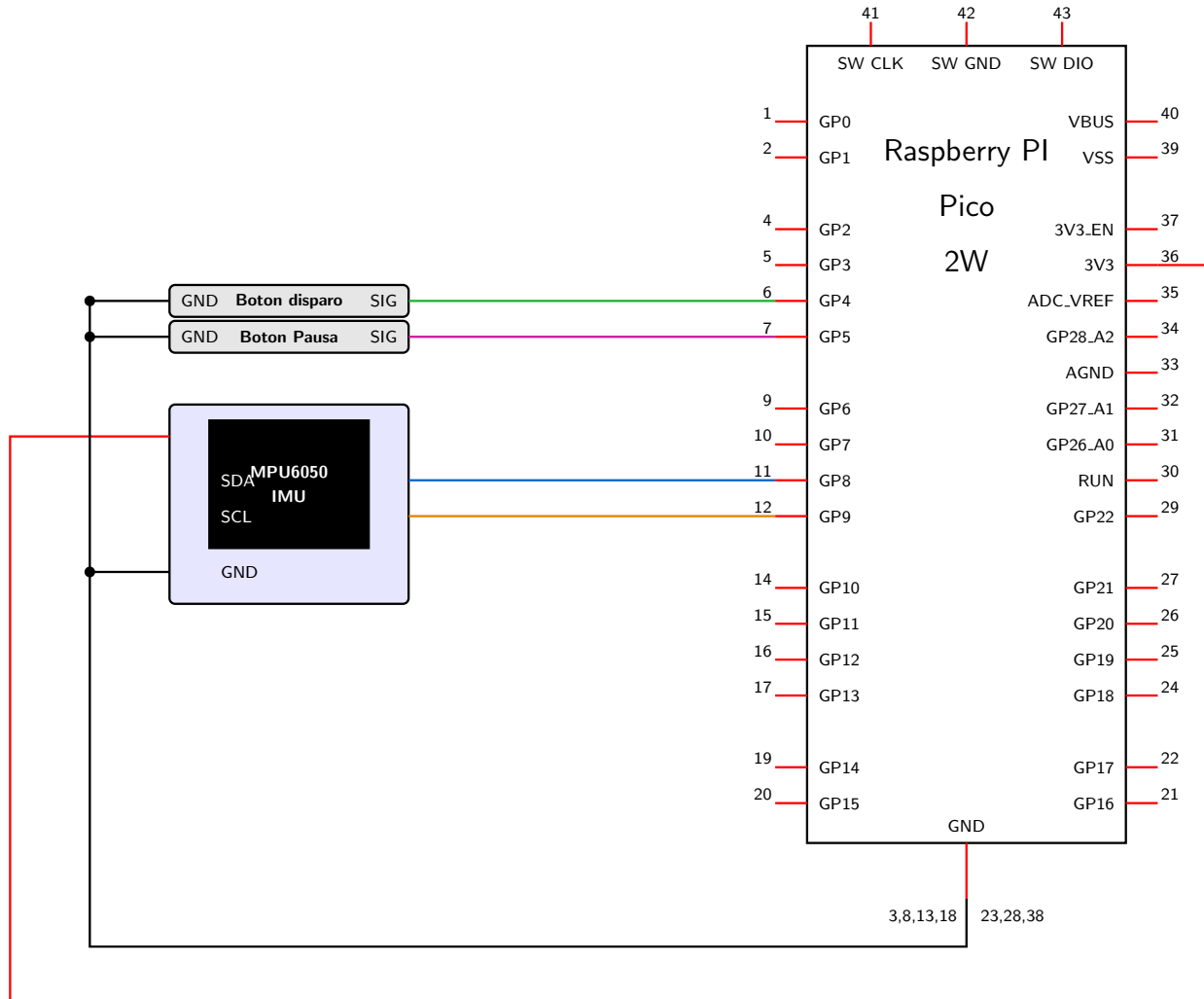
recibe su alimentación y la iluminación del *backlight* desde el pin 3V3.

Tabla 3: Conexiones del módulo principal (Consola)

Periférico	Pin Periférico	Pin Pico 2W	Descripción
TFT ILI9341	VCC	3V3 (Pin 36)	Alimentación 3.3 V
TFT ILI9341	GND	GND (Pin 38)	Tierra
TFT ILI9341	CS	GP17 (Pin 22)	Chip Select SPI
TFT ILI9341	RESET	GP21 (Pin 27)	Reset de display
TFT ILI9341	DC	GP20 (Pin 26)	Dato / Comando
TFT ILI9341	MOSI	GP19 (Pin 25)	Datos SPI (salida)
TFT ILI9341	SCK	GP18 (Pin 24)	Reloj SPI (40 MHz)
TFT ILI9341	LED	3V3 (Pin 36)	Backlight (compartido)
TFT ILI9341	MISO	GP16 (Pin 21)	Datos SPI (entrada)
Buzzer	SIG	GP15 (Pin 20)	Señal PWM de audio
Buzzer	GND	GND (Pin 38)	Tierra

El bus SPI se configura a 40 MHz de frecuencia de reloj con polaridad y fase en 0 (modo SPI 0). La clase `Display` del driver ILI9341 maneja internamente los comandos de inicialización de la pantalla y expone los métodos `draw_sprite()`, `fill_hrect()`, `draw_pixel()` y `draw_text8x8()` que utiliza el motor de juego.

4. Diagrama de conexiones del mando



El mando integra el sensor IMU MPU-6050 para la detección de movimiento por inclinación y dos botones físicos con resistencias de *pull-up* internas habilitadas por software. El MPU-6050 se conecta al bus I2C0 de la Pico utilizando los pines GP8 (SDA) y GP9 (SCL) a una frecuencia de 400 kHz. Los botones se conectan directamente entre el pin GPIO y GND, aprovechando las resistencias de *pull-up* internas del RP2350. La alimentación del MPU-6050 se toma del pin 3V3 de la Pico y se enruta externamente por debajo del circuito para evitar cruces con las señales de datos.

Tabla 4: Conexiones del módulo del mando

Periférico	Pin Periférico	Pin Pico 2W	Descripción
MPU-6050	VCC	3V3 (Pin 36)	Alimentación 3.3 V
MPU-6050	GND	GND (Pin 38)	Tierra
MPU-6050	SDA	GP8 (Pin 11)	Dato I2C (400 kHz)
MPU-6050	SCL	GP9 (Pin 12)	Reloj I2C (400 kHz)
Botón Disparo	SIG	GP4 (Pin 6)	Entrada con Pull-up interno
Botón Disparo	GND	GND (común)	Tierra
Botón Pausa	SIG	GP5 (Pin 7)	Entrada con Pull-up interno
Botón Pausa	GND	GND (común)	Tierra

El software del mando (`main.py` del mando) lee las aceleraciones del MPU-6050 utilizando la librería `imu` (clase `MPU6050` sobre I2C), muestrea el estado de los botones en cada iteración del bucle asíncrono y construye la trama BLE de 10 bytes que se transmite vía notificación GATT cada 10 ms. La gestión de memoria se optimiza con llamadas periódicas a `gc.collect()` para evitar fragmentación en el heap de MicroPython durante la ejecución continua.

4.1. Protocolo BLE

Ambos microcontroladores utilizan el módulo `aioble` sobre el stack Bluetooth integrado del chip RP2350 de la Pico 2W. El servicio GATT se define con UUIDs de 128 bits personalizados para evitar conflictos con servicios estándar. La característica de datos se configura con permisos de lectura y notificación en el periférico. La Consola realiza un escaneo activo de 5 segundos buscando el nombre de dispositivo `'PicoMando'`; al encontrarlo, establece la conexión, descubre el servicio y se suscribe a las notificaciones. Ante cualquier desconexión o error, el task BLE espera 2 segundos y reintenta automáticamente, garantizando la reconexión sin intervenir la lógica de juego.

```
1 _SERVICE_UUID = bluetooth.UUID('12345678-1234-1234-1234-123456789abc')
2 _CHAR_UUID     = bluetooth.UUID('87654321-4321-4321-4321-cba987654321')
```

5. Resultados:

5.1. Prueba de Comunicación y Enlace BLE

El primer resultado del sistema fue validar que se estableció correctamente la comunicación inalámbrica entre los dos microcontroladores. Para comprobar la correcta ejecución de las tareas, se monitoreó la salida de los dispositivos a través de la terminal del entorno de desarrollo Thonny.

En la siguiente imagen se presenta la captura de los registros generados durante la prueba de emparejamiento. Se ve el momento en que el escaneo activo de la consola esta buscando el mando y cuando lo localiza, establece la conexión.

```
28 class ControllerState:
29     def __init__(self):
30         self.x = 0.0
31         self.y = 0.0
```

Consola x

```
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
Buscando Mando Invaderoids...
|Mando Encontrado! Conectando...
|Conectado! Recibiendo datos...
```

Figura 1: Terminal de Thonny confirmando el establecimiento exitoso de la conexión Bluetooth.

5.2. Experiencia en el juego

5.2.1. Pantalla de inicio

Una vez que la conexión Bluetooth se establecio correctamente, la pantalla de la consola muestra la interfaz de inicio. En la siguiente imagen se aprecia la portada del juego, donde incluimos el nombre de la universidad, la materia y los integrantes del equipo. El sistema se queda a la espera de que el jugador presione el botón de disparo en el mando para comenzar la partida.

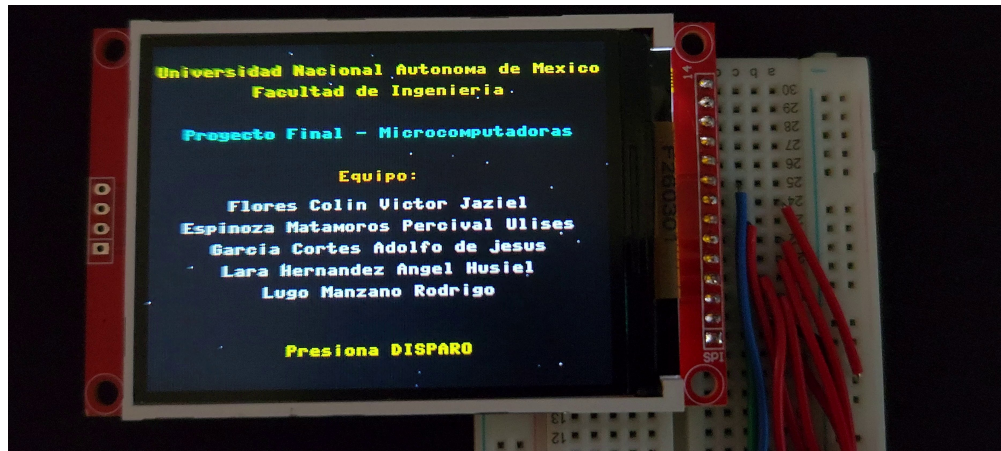


Figura 2: Pantalla principal de inicio esperando la interacción del jugador.

5.2.2. Pantalla de información

Inmediatamente después de la pantalla de inicio y antes de arrancar a jugar como tal la partida, el sistema muestra una pantalla con información del juego. En la siguiente imagen se observan los distintos tipos de enemigos junto con la puntuación que otorgan al ser destruidos: 50, 30, 20 y 10 dependiendo de la nave. También se muestran los obstáculos, como el meteorito cuyo valor aparece como “??” ya que te penaliza quitando de 1 a 2 puntos dependiendo de si le disparas o lo dejas pasar, y la nave aliada que te otorga una vida extra (+1).



Figura 3: Pantalla de información con los puntajes de cada enemigo y obstáculo.

5.2.3. Inicio del desarrollo de la partida

Al iniciar el juego, en la siguiente imagen podemos ver cómo se despliega los enemigos en la parte superior y la nave de jugador en la inferior. También se ve el fondo dinámico de estrellas y los marcadores de puntuación, nivel y vidas restantes en los bordes superiores de la pantalla. Durante las pruebas notamos que el movimiento de la nave se siente muy fluido y la rapidez va de acuerdo al grado de inclinación del control y la pantalla no presenta parpadeos molestos gracias a la optimización de borrado que implementamos.



Figura 4: Primera fase del juego con la formación de enemigos y la interfaz en pantalla.

5.2.4. Menú de pausa

Si el jugador presiona el botón de pausa durante el juego, el sistema detiene la acción y despliega dicho menú. En la foto se muestra esta pantalla, donde el jugador puede elegir entre continuar la partida, cambiar la dificultad o salir al menú principal. Comprobamos que la navegación por las opciones usando la inclinación del mando funciona a la perfección.



Figura 5: Menú de pausa interactivo controlado por el movimiento del mando.

5.2.5. Colisiones y pérdida de vidas

La detección de colisiones fue otro punto clave que evaluamos. En la siguiente secuencia de dos imágenes capturamos el momento exacto en que la nave del jugador es impactada. Primero se ve la explosión generada por el choque, y en la segunda imagen se observa cómo la nave reaparece en el centro (parpadeando), para indicar un breve periodo de invulnerabilidad, mientras el contador de vidas disminuye correctamente.



Figura 6: Momento exacto del impacto mostrando la animación de explosión de la nave.



Figura 7: Reaparición del jugador con invulnerabilidad temporal y la actualización del contador de vidas.

5.2.6. Progresión de niveles

Conforme el jugador elimina a todos los enemigos de la pantalla, el sistema avanza al siguiente nivel. En las siguientes tres imágenes se ve esta transición. Primero se muestra el nivel completado sin enemigos, luego el mensaje de preparación para el siguiente asalto, donde la velocidad y la dificultad aumentan.



Figura 8: Nivel sin enemigos.



Figura 9: Mensaje de transición hacia el siguiente nivel.

5.2.7. Fin del juego

Finalmente, cuando el jugador pierde todas sus vidas, se detecta el final de la partida. En la siguiente imagen se ve la pantalla de fin de juego, la cual muestra la puntuación final obtenida y el nivel máximo alcanzado. Desde aquí, el sistema espera nuevamente a que se presione el botón de disparo para reiniciar todo el ciclo de juego.



Figura 10: Pantalla de fin de juego con el resumen de la puntuación.



6. Conclusiones:

- **Espinoza Matamoros Percival Ulises:**
- **Flores Colin Victor Jaziel:**
- **García Cortés Adolfo de Jesús:** El desarrollo de este proyecto me permitió aplicar de manera práctica los conocimientos adquiridos durante el curso de microcomputadoras haciendo un videojuego estilo Galaga completamente funcional utilizando microcontroladores Raspberry Pi Pico 2W. Recrear la experiencia y las mecánicas principales de este clásico implicó integrar correctamente tanto el hardware como el software.

En la parte de conectividad, se logró establecer una comunicación estable mediante Bluetooth Low Energy (BLE) usando una arquitectura cliente-servidor. Gracias a esto, una segunda tarjeta pudo funcionar como control inalámbrico, enviando en tiempo real los datos del sensor MPU-6050 mediante I2C y las señales de los botones físicos. Esto permitió que el movimiento de la nave se sintiera rápido y preciso durante la partida.

En el apartado de software, uno de los aprendizajes más importantes fue el uso de programación asíncrona con uasyncio. Esta estructura permitió que diferentes tareas funcionaran al mismo tiempo sin interferirse entre sí, como la comunicación BLE, el movimiento de los enemigos, la detección de colisiones y la generación de sonido mediante PWM. Además, se optimizó el rendimiento gráfico usando renderizado diferencial por SPI, redibujando únicamente las partes de la pantalla TFT que cambiaban. Gracias a esto, se eliminó el parpadeo visual y se mantuvo una experiencia de juego fluida.

En conclusión, la construcción de este Galaga representó un reto. El proyecto me permitió aplicar y reforzar conocimientos sobre programación de bajo nivel, manejo de protocolos de comunicación como SPI, I2C y BLE, así como el manejo eficiente de los recursos del microcontrolador.

- **Lara Hernandez Angel Husiel:**
- **Lugo Manzano Rodrigo:**